# Plan: Challenge IA — Sistema de reservas de laboratorio con agentes

## Contexto

La consigna (challenge-agents.pdf) pide un sistema local de reservas para 5 unidades físicas (MIC-01, MIC-02, OSC-01, MUL-01, FUE-01), construido con un agente de programación real. Se evalúa la **evidencia en el repositorio**:

- Especificación: 25 %
- Preparación y uso del agente: 25 %
- Funcionamiento y pruebas: 30 %
- Trazabilidad, con 3 episodios reales de delegación: 20 %

La carpeta hoy solo tiene el PDF. Todavía no hay repositorio Git.

**Decisiones ya tomadas con el usuario:**

- Stack: Python 3.14, SQLite (`sqlite3` de la biblioteca estándar) y pytest.
- Interfaz: CLI de consola.
- Agentes: Claude Code como agente principal, más subagentes en `.claude/agents/`.
- Repositorio: `git init` local. El usuario crea el repo en GitHub y hace el push al final.

## Agentes (harness: Claude Code, modelo `claude-opus-5-5`)

| Agente | Archivo | Rol | Herramientas |
|---|---|---|---|
| Principal | `CLAUDE.md` | Implementa, coordina y mantiene `docs/progreso.md` | todas, con aprobación humana |
| `redactor-pruebas` | `.claude/agents/redactor-pruebas.md` | Escribe pruebas a partir de la especificación (criterios CA-xx), sin mirar la implementación | Read, Glob, Grep, Write, Edit (solo `tests/`) |
| `revisor-reglas` | `.claude/agents/revisor-reglas.md` | Audita el código contra las reglas de negocio y genera hallazgos. No edita | Read, Glob, Grep |
| `verificador` | `.claude/agents/verificador.md` | Corre pytest y el procedimiento de persistencia, y registra comando, versión y salida real | Read, Bash, Write (solo `docs/verificacion.md`) |

**Adaptación específica comprobable**, escrita en `CLAUDE.md`:

**1.** Regla explícita de intervalo semiabierto `[inicio, fin)`. Un fin igual a otro inicio no se superpone. Toda consulta de disponibilidad usa la misma función `se_superpone()`.

**2.** Procedimiento de persistencia: siempre se escribe en una transacción, nunca se borran reservas y cancelar solo cambia `estado`.

**3.** Al cerrar cada etapa se actualiza `docs/progreso.md` y se hace commit.

`.claude/settings.json` permite `pytest`, `python` y `git status/diff/log`. Pide aprobación para `git commit`, para borrar archivos y para cualquier acceso a red.

## Especificación (`docs/especificacion.md`)

**Requisitos:**

- R1: inventario precargado.
- R2: consultar disponibilidad.

- R3: crear reserva.
- R4: cancelar.
- R5: persistencia.
- R6: historial y listado.

**Reglas:**

- Unidad conocida.
- Solicitante no vacío ni compuesto solo por espacios. Se le aplica `strip`.
- Fechas válidas e inicio estrictamente anterior al fin.
- Una superposición solo se rechaza entre reservas activas de la misma unidad.
- Se permiten límites consecutivos.
- Se permiten reservas simultáneas de unidades distintas.

**Supuestos propuestos (a revisar):**

- Zona horaria: hora local del laboratorio (America/Argentina/Buenos_Aires), guardada sin offset.
- Formato de fecha y hora: `YYYY-MM-DD HH:MM`, con minutos como resolución.
- Fechas pasadas: no se puede crear una reserva con inicio en el pasado. El "ahora" se inyecta para que las pruebas sean deterministas.
- Duración máxima: 12 h. No hay límite de anticipación.
- Una reserva ya cancelada no se puede volver a cancelar (se informa error).
- IDs con formato `R-0001`, autoincremental.

**Criterios de aceptación CA-01…CA-12:** cada uno en formato dado/cuando/entonces e incluye el ejemplo de la consigna (MIC-01 10:00–11:00).

## Estructura de la solución

```
README.md                  # 4 secciones: Especificación, Solución, Preparación del agente, Decisiones y delegación
CLAUDE.md
.claude/settings.json  .claude/agents/*.md
pyproject.toml  requirements-dev.txt (versiones fijadas = lock)
data/inventario.json       # inventario inicial versionado
src/reservas/
  dominio.py      # Reserva, Estado, validaciones, se_superpone()
  inventario.py   # carga de inventario.json
  repositorio.py  # SQLite: esquema, insertar, cancelar, listar; transacción BEGIN IMMEDIATE
  servicio.py     # disponibilidad / reservar / cancelar (mismas reglas)
  cli.py          # argparse: inventario, disponibilidad, reservar, cancelar, listar, reiniciar-datos --confirmar
tests/  test_crear.py test_superposicion.py test_limites.py test_unidades.py
        test_validaciones.py test_cancelacion.py test_persistencia.py (subproceso: 2 ejecuciones del CLI sobre la misma BD)
docs/  especificacion.md progreso.md decisiones.md (ADR + 3 episodios) agente.md verificacion.md
```

- La base de datos vive en `data/reservas.db`, que queda ignorada por git. Se puede cambiar la ruta con `--db` o con `RESERVAS_DB`.
- `reiniciar-datos` **borra las reservas**. Exige `--confirmar` y la advertencia figura en el README.

## Etapas de trabajo

Cada etapa termina en un commit y en una actualización de `progreso.md`. Las marcadas como [Checkpoint] son checkpoints donde vos revisás el resultado.

**-1. Exportar el plan:** se guarda en `docs/plan/plan-trabajo.md` y se genera `plan-trabajo.docx` (con python-docx) y `plan-trabajo.pdf` (con reportlab), las dos librerías instaladas en un venv de herramientas que no forma parte del entregable.

**0. Preparación:** `git init`, `.gitignore`, `CLAUDE.md`, settings, los 3 subagentes y `progreso.md`. → commit `chore: preparación del agente`

**1. Especificación v1** → commit. [Checkpoint] Revisás los supuestos y criterios. Cualquier cambio que pidas va a un commit `spec v2` que explica el motivo. Así la evolución queda recuperable.

**2. Episodio 1 (implementación):** el agente principal implementa dominio, repositorio, servicio y CLI. [Checkpoint] Aceptás o corregís.

**3. Episodio 2 (pruebas):** `redactor-pruebas` escribe las pruebas a partir de los CA, se ejecutan y se registran los fallos reales que aparezcan. [Checkpoint]

**4. Episodio 3 (revisión):** `revisor-reglas` audita el código contra las reglas. Vos decidís qué hallazgos aceptar o rechazar y se aplican las correcciones. [Checkpoint]

**5. Verificación:** `verificador` corre `pytest -v` y el procedimiento manual de persistencia (reservar, cancelar, salir, reiniciar y listar). Anota los resultados esperados y los observados en `verificacion.md`.

**6. Documentación:** README con las 4 secciones, `decisiones.md` con los 3 episodios (objetivo → contexto e instrucción → resultado del agente → decisión humana → enlace a commit o prueba) y `agente.md` con el entorno, `claude --version`, el modelo, los permisos y lo que se excluyó.

**7. Cierre:** commit final y las instrucciones para hacer el push y enviar el mail a dgaifirst@gmail.com con el asunto «Challenge IA».

**Regla de honestidad:** los episodios registran solo lo que ocurrió. Las decisiones que tomes en cada [Checkpoint] se anotan con tus palabras. No se inventan fallas.

## Verificación

- `python -m venv .venv && pip install -r requirements-dev.txt && pip install -e .`
- `pytest -v`: cubre creación válida, superposición, límites consecutivos, unidades distintas, entradas inválidas, cancelación que conserva el registro y libera el horario, y persistencia.
- Prueba manual de punta a punta: `reservas reservar MIC-01 "Ana Ficticia" "2026-12-01 10:00" "2026-12-01 11:00"`. Después se prueba `10:30–11:30`, que debe rechazarse, y `11:00–12:00`, que debe aceptarse. Luego se reserva MIC-02 de 10:00 a 11:00, que debe aceptarse. Se cancela, se cierra la aplicación, se vuelve a abrir y con `listar` se comprueba que figuran las reservas activas y la cancelada.